Lecture 27: Arrays in MIPS Assembly Language

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand how arrays are stored in memory.
- Declare integer and byte arrays.
- Access array elements using addresses and offsets.
- Traverse arrays using loops.
- Read and modify array elements.
- Calculate the sum of an array.
- Find the maximum and minimum values in an array.
- Debug array programs in the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how arrays are stored in memory, how to access and traverse them in MIPS Assembly Language, and how to write programs that calculate sums and find maximum and minimum values using the MARS simulator."**

So far, we've learned how to work with variables, perform arithmetic operations, use memory instructions, make decisions with branch instructions, and repeat tasks using loops.

But in real-world programming, we rarely work with just one value. Instead, we often need to store many related values together, such as student marks, employee salaries, temperatures, or sensor readings.

That's where arrays come in.

In today's lecture, we'll learn how arrays are stored in memory, how to access individual elements, and how to process arrays using loops in MIPS Assembly Language.

By the end of this lecture, you'll be able to write programs that calculate sums, search through arrays, and modify array elements in the MARS simulator.

Let's begin.

---

2. What is an Array?

An **array** is a collection of elements of the same data type stored in **contiguous memory locations**.

Example in C++:

int marks[5] = {10,20,30,40,50};

Equivalent in MIPS:

.data

marks: .word 10,20,30,40,50

---

3. Memory Representation of an Array

Each `.word` occupies **4 bytes**.

Example:

marks: .word 10,20,30,40,50

Memory layout:

Address	Value
10010000	10
10010004	20
10010008	30
10010012	40
10010016	50

Notice:

Every next element is **4 bytes** after the previous one.

---

4. Array Index Formula

For an integer array:

Address of Element

=

Base Address + (Index × 4)

Example:

marks[3]

Base = 10010000

3 × 4 = 12

Address = 10010012

This formula is extremely important for exams.

---

5. Loading the Base Address

Before accessing an array:

la $s0, marks

Now:

$s0

↓

Base Address of Array

Remember:

- `la` loads the **address**.
- `lw` loads the **value**.

---

6. Accessing the First Element

la $s0, marks

lw $t0,0($s0)

Result:

$t0 = 10

---

7. Accessing Other Elements

Second element:

lw $t1,4($s0)

Third element:

lw $t2,8($s0)

Fourth element:

lw $t3,12($s0)

Fifth element:

lw $t4,16($s0)

---

8. Modifying an Array Element

Suppose we want:

marks[2] = 100

MIPS:

li $t0,100

sw $t0,8($s0)

Memory becomes:

10

20

100

40

50

---

9. Traversing an Array Using a Loop

Instead of writing:

lw $t0,0($s0)

lw $t0,4($s0)

lw $t0,8($s0)

...

We use a loop.

---

10. Loop Variables

Use:

$t0 → Index

$t1 → Address

$t2 → Current Element

---

11. Program – Print Every Array Element

```
.data

array: .word 5,10,15,20,25

.text

.globl main

main:

    la $s0,array

    li $t0,0
```

```
Loop:

    slti $t3,$t0,5

    beq $t3,$zero,Exit

    sll $t1,$t0,2

    add $t1,$t1,$s0

    lw $t2,0($t1)

    li $v0,1
    move $a0,$t2
    syscall

    li $v0,11
    li $a0,10
    syscall

    addi $t0,$t0,1
```

```
    j Loop

Exit:

li $v0,10
syscall
```

Output:

```
5

10

15

20

25
```

---

12. Why Use `sll`?

To calculate:

Index × 4

Instead of multiplication:

mul $t1,$t0,4

We use:

sll $t1,$t0,2

because shifting left by **2 bits** multiplies by **4**, and it is more efficient.

Examples:

1 << 2 = 4

2 << 2 = 8

3 << 2 = 12

---

13. Program – Sum of an Array

.data

array: .word 10,20,30,40,50

.text

```
main:

la $s0,array

li $t0,0

li $t4,0

Loop:

slti $t3,$t0,5

beq $t3,$zero,Finish

sll $t1,$t0,2

add $t1,$t1,$s0

lw $t2,0($t1)

add $t4,$t4,$t2
```

addi $t0,$t0,1

j Loop

Finish:

li $v0,10

syscall

Final result:

$t4 = 150

---

14. Program – Find the Maximum Element

Logic:

max = first element

Loop

if(current > max)

max = current

Pseudo MIPS:

lw max

Loop

lw current

if current > max

move max,current

Explain the algorithm first before coding it.

---

15. Program – Find the Minimum Element

Same logic:

min = first element

Loop

if(current < min)

min = current

---

16. Byte Arrays

Characters:

letters:

.byte 'A','B','C','D'

Access:

lb $t0,0($s0)

Notice:

- `.word` → `lw`
- `.byte` → `lb`

---

17. Real-Life Analogy

Imagine lockers in a school.

Locker 0

Locker 1

Locker 2

Locker 3

Every locker has an address.

The array works exactly the same way.

---

18. Debugging Arrays in MARS

Demonstrate:

1. Open **Data Segment**.
2. Observe array values.
3. Step through the program.
4. Watch:
    - `$t0` (index)
    - `$t1` (address)
    - `$t2` (element)
5. Observe how addresses increase by **4 bytes**.

This visualization makes array traversal much easier to understand.

---

19. Common Mistakes

Students often:

❌ Forget to multiply the index by 4.

Incorrect:

add $t1,$s0,$t0

Correct:

sll $t1,$t0,2

add $t1,$t1,$s0

❌ Use `lw` for byte arrays.

Correct:

lb

❌ Forget to load the array base address.

Always start with:

la $s0,array

🎤 Lecture Summary

Today, we learned how arrays are represented and processed in MIPS Assembly Language.

We declared integer and byte arrays, accessed elements using base addresses and offsets, traversed arrays with loops, and wrote programs to print elements, calculate

sums, and determine the maximum and minimum values. We also learned why `sll` is commonly used to calculate array offsets efficiently.

In the next lecture, we'll study **Strings in MIPS Assembly Language**, including string declaration, input and output, character-by-character processing, and common string operations such as finding the length, copying, and comparing strings.

Thank you for watching, and I'll see you in the next lecture.
